

Agile Requirements: Opportunity or Oxymoron?

Ken Orr

Although some might argue, “We don’t need requirements because we’re agile,” can we really say that agility and requirements are mutually exclusive? Ken Orr shows us that, in today’s world, requirements without agility is just as bad as agility without requirements.

—Suzanne Robertson

Requirements is a favorite topic of mine, not because it’s so difficult, but because it’s so often neglected. In discussions with colleagues, I’ve discovered a consensus that systems analysis and requirements definition, in particular, have fallen on hard times. Ten or 15 years ago we taught people structured analysis, information engineering, data modeling, and working with computer-aided software engineering tools to turn those requirements into running programs. For some reason, we quit emphasizing requirements and decided to take another tack. Only recently, with the increased emphasis on the Unified Modeling Language (UML) and Model Driven Architecture, does the industry seem to be finally returning to where it left off a decade or so ago.



Back to basics

I have some guesses why requirements definition has been so neglected. Mostly, I think, it’s because “doing requirements” requires spending more time studying the business and application world and less time programming. Doing requirements takes time—time that could be spent writing code—and users often feel better if they think that people are “working” (that is, coding) on their problems.

Additionally, some agile development advocates suggest that requirements aren’t really

necessary in an agile environment; in the new agile world, you can code yourself into success. Indeed, for many in the agile world, the code is the requirements. Rather than understanding the user’s business and information needs, some agile approaches attempt to give users what they say they want as quickly as possible.

But these aren’t reasons for not doing requirements, they’re excuses. Every system of consequence needs good requirements. Without good requirements, the project risks increase dramatically. The better the requirements, the better people understand what they’re trying to build (or acquire). But requirements aren’t—or shouldn’t be—measured in terms of length. Bigger is not necessarily better. Requirements should define something that you have to build or otherwise acquire. If it doesn’t, it’s not an engineering document, it is a political one. (A couple of years ago, there was a made-for-TV movie called “The Pentagon Wars” about the development of the Bradley Fighting Vehicle. A particularly funny 10-minute segment showed how the requirements grew from an unarmed, armored transport for 13 soldiers to a highly armed, lightly (aluminum) armored transport for six people that was so dangerous that the army was afraid to test it. With additional “requirements” loaded on by one of the armed services to suit the ego of its representative, the vehicle became less and less suitable for its primary function.)

Defining requirements shouldn’t be difficult, but it often is. I think this is true because those

of us who teach requirements or requirements engineering haven't clarified what "doing requirements" really amounts to because, I suppose, it isn't really clear to us.

Outcomes and constraints

Some years ago, some colleagues and I came up with what we considered the clearest definition of what requirements were (or ought to be). We concluded that there were really only two kinds of requirements: systems outputs (outcomes) and constraints. Some people call this approach output-oriented requirements definitions, but I prefer to call it teleological (goal-oriented) requirements.

In this era of increasingly complex technological solutions, this definition seems overly simplistic, but it works. What we've found is that if you don't know your system's primary outputs, you simply aren't finished with your requirements. And if you do know what your primary outputs are (and this includes data attributes, the output's structure, and any decision or calculation rules), you're in a pretty good place. Everything besides primary outputs are constraints—things you have to do while you're producing the primary outputs (for example, processing some number of simultaneous users, processing some number of transactions per hour, providing some minimal response time, and so on).

Getting the primary outputs right is ultimately more important than getting the constraints right. And you can distinguish between a primary output and a constraint by asking a simple question: "If x went away, would we still do the system?" Take a payroll system that produces payroll checks and payroll information for annual tax purposes (in the US, the W2 form), for example. Which of these do you suppose is a primary output? The answer is, of course, the payroll check (or some electronic form thereof). Clearly, if the payroll check went away, the system would stop producing the W2 forms. On the other hand, if the system no longer needed to produce the W2 form,

it would still need to produce payroll checks. Simple, huh?

So, my basic view is that requirements engineers should focus first on the business model to identify the basic results (outputs) and secondly on constraints. Unfortunately, in most organizations it happens the other way around. Indeed, most requirements that go into a typical RFP (request for proposal) are constraints. These constraints might be requirements, but they don't define the system's core purpose. Unfortunately, many systems, especially large government systems, are both overconstrained and underspecified.

Simplifying the requirements down to outputs and constraints has a major real-world advantage—you can test them. (Testability, as you might imagine, is particularly useful for big systems.) In fact, you can reasonably ask whether anything you can't test is really a requirement at all.

Making the right map

If all this sounds too easy, it is. As it turns out, most systems users don't know what they want out of a system or what they could get. So, before you can identify the primary outputs, you have to understand such things as the business context, business process, and the information users need to perform key activities in the business process. The requirements engineer's job is to help the users discover what they really need. Furthermore, because in any significant system there's never just one user, coming up with an entire system's minimal outputs

involves combining subject matter expertise from several experts (users). Not surprisingly, reaching consensus with all those users can be hard work.

In my office, I have several posters with the same theme. The archetype is the "New Yorker's View of the World"—you know, the one where Manhattan and the Bronx are really big and everything else is really small. And I have similar posters for Atlanta, Chicago, Denver, and San Francisco. In each one, you get a biased view; everything local is important (really big) and everything else is trivial (really small).

Those posters are there to remind me of how users see the world. The requirements engineer's job, then, is much like that of a map maker trying to create a good map from all those individual viewpoints. First you need to gather all the (distorted) individual views, and then you need to combine them to create a coherent whole. Beginning requirements engineers believe the user is always right, but most users are, at best, only partly right.

Some current requirements approaches can help. These approaches, while not foolproof, usually give good requirements. Some of the current ones are better than nothing. I don't think too much of some of the UML stuff, mostly because it's driven more from the technology side than the business side. Use case and sequence diagrams don't do a great job, I find, of communicating with end users, but they're certainly better than not doing requirements at all.

Inventing great requirements

As I've mentioned, getting requirements isn't that difficult, but getting great requirements is much harder—great requirements require imagination in working with users. James Robertson wrote a fantastic piece for this series last year entitled "Eureka! Why Analysts Should Invent Requirements." In that article, he suggested that rather than taking a passive role of just writing down what users ask for, requirements analysts should help "invent requirements." And, of course, he's right. Not all a proposed system's outputs

The requirements engineer's job is to help the users discover what they really need.

and constraints exist in the current users' minds. In many cases, the key users don't know what's possible, they don't know what the competition is doing, and they don't know what the state of the art is. So, someone needs to help them think ahead.

People don't hire good architects to get so-so buildings, and people shouldn't hire good requirements analysts to produce so-so systems. I watched a movie recently in which a fellow blurts out to the love of his life, "You're everything I never knew I always wanted." Great architecture is like that; great products are like that; great requirements are like that.

To get great requirements, you don't have to be a genius or a magician. You just have to know your users' outputs and constraints and then document those needs in a way they understand.

Not an oxymoron

If agile developers insist on directly

involving users and delivering "product versions" in short increments, does that mean there's no place for requirements in the new agile world? The short answer is no, but this means the agile movement will have to look at its prejudice against design tools. I suggest using prototypes, which are a great way to improve the definition process. In most circumstances, prototypes are the absolute best way to get users engaged in defining their outputs and constraints. Other fields—for example, architecture, electrical engineering, product engineering, and so on—have combined rigorous requirements and design with automated tools that produce prototypes to dramatically shorten the time it takes to move from concept to finished products. They haven't abandoned the requirements process, they've abandoned the laborious process of hand design and fabrication.

It's both possible and necessary to combine requirements and agile development. Ours is an impatient age, but there's no reason we can't have agile requirements, with all the powerful hardware and sophisticated graphic software available. Today we can capture business models, workflows, activities, and data structures quickly and in enough detail to automatically generate database designs and code that actually produce prototypes and products as rapidly as CAD/CAM tools do in architecture and engineering. When people realize how close we are, the requirements pendulum should move back toward understanding business and user needs. ☞

Ken Orr is the chief scientist of the Ken Orr Institute and the developer of the Warnier-Orr Methodology. Contact him at kenorr@kenorrinst.com.

PURPOSE The IEEE Computer Society is the world's largest association of computing professionals, and is the leading provider of technical information in the field.

MEMBERSHIP Members receive the monthly magazine *Computer*, discounts, and opportunities to serve (all activities are led by volunteer members). Membership is open to all IEEE members, affiliate society members, and others interested in the computer field.

COMPUTER SOCIETY WEB SITE

The IEEE Computer Society's Web site, at www.computer.org, offers information and samples from the society's publications and conferences, as well as a broad range of information about technical committees, standards, student activities, and more.

BOARD OF GOVERNORS

Term Expiring 2004: Jean M. Bacon, Ricardo Baeza-Yates, Deborah M. Cooper, George V. Cybenko, Harubisha Ichikawa, Thomas W. Williams, Yervant Zorian

Term Expiring 2005: Oscar N. Garcia, Mark A. Grant, Michel Israel, Stephen B. Seidman, Kathleen M. Swigger, Makoto Takizawa, Michael R. Williams

Term Expiring 2006: Mark Christensen, Alan Clements, Annie Combelles, Ann Gates, Susan Mengel, James W. Moore, Bill Schilit

Next Board Meeting: 12 June 2004, Long Beach, CA

IEEE OFFICERS

President: ARTHUR W. WINSTON

President-Elect: W. CLEON ANDERSON

Past President: MICHAEL S. ADLER

Executive Director: DANIEL J. SENESE

Secretary: MOHAMED EL-HAWARY

Treasurer: PEDRO A. RAY

VP, Educational Activities: JAMES M. TIEN

VP, Pub. Services & Products: MICHAEL R. LIGHTNER

VP, Regional Activities: MARC T. APTER

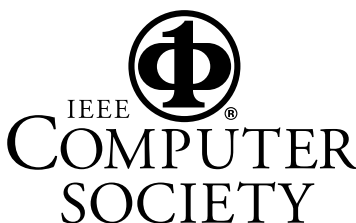
VP, Standards Association: JAMES T. CARLO

VP, Technical Activities: RALPH W. WYNDRUM JR.

IEEE Division V Director: GENE H. HOFFNAGLE

IEEE Division VIII Director: JAMES D. ISAAK

President, IEEE-USA: JOHN W. STEADMAN



COMPUTER SOCIETY OFFICES

Headquarters Office

1730 Massachusetts Ave. NW
Washington, DC 20036-1992
Phone: +1 202 371 0101
Fax: +1 202 728 9614
E-mail: bq.ofc@computer.org

Publications Office

10662 Los Vaqueros Cir., PO Box 3014
Los Alamitos, CA 90720-1314
Phone: +1 714 821 8380
E-mail: help@computer.org

Membership and Publication Orders:

Phone: +1 800 272 6657
Fax: +1 714 821 4641
E-mail: help@computer.org

Asia/Pacific Office

Watanabe Building
1-4-2 Minami-Aoyama, Minato-ku
Tokyo 107-0062, Japan
Phone: +81 3 3408 3118
Fax: +81 3 3408 3553
E-mail: tokyo.ofc@computer.org



EXECUTIVE COMMITTEE

President:

CARL K. CHANG*
Computer Science Dept.
Iowa State University
Ames, IA 50011-1040
Phone: +1 515 294 4377
Fax: +1 515 294 0258
c.chang@computer.org

President-Elect: GERALD L. ENGEL*

Past President: STEPHEN L. DIAMOND*

VP, Educational Activities: MURALI VARANASI*

VP, Electronic Products and Services:

LOWELL G. JOHNSON (1ST VP)*

VP, Conferences and Tutorials:

CHRISTINA SCHÖBER*

VP, Chapters Activities:

RICHARD A. KEMMERER (2ND VP)†

VP, Publications: MICHAEL R. WILLIAMS†

VP, Standards Activities: JAMES W. MOORE†

VP, Technical Activities: YERVANT ZORIAN†

Secretary: OSCAR N. GARCIA*

Treasurer: RANGACHAR KASTURI†

2003-2004 IEEE Division V Director:

GENE H. HOFFNAGLE†

2003-2004 IEEE Division VIII Director:

JAMES D. ISAAK†

2004 IEEE Division VIII Director-Elect:

STEPHEN L. DIAMOND*

Computer Editor in Chief: DORIS L. CARVER†

Executive Director: DAVID W. HENNAGE†

* voting member of the Board of Governors

† nonvoting member of the Board of Governors

EXECUTIVE STAFF

Executive Director: DAVID W. HENNAGE

Assoc. Executive Director: ANNE MARIE KELLY

Publisher: ANGELA BURGESS

Assistant Publisher: DICK PRICE

Director, Finance & Administration:

VIOLET S. DOAN

Director, Information Technology & Services:

ROBERT CARE

Manager, Research & Planning: JOHN C. KEATON